LUUMU · PLUGIN DE ACESSIBILIDADE

Integração com plataforma externa

O que enviar ao SDK para exibir o convite certo e como receber, em tempo real, cada ajuste feito pelo estudante.

Este documento é para o time técnico da plataforma que controla os estudantes (quem é convidado e onde ficam salvas as escolhas de acessibilidade). A Luumu exibe o botão, o painel de recursos e o convite dentro da página; a plataforma externa decide e guarda.

1. Visão geral

1. O estudante entra na plataforma. A página chama `Luumu.accessibility.setUser({...})` com o que a plataforma sabe dele: se recebe o convite, quais perfis sugerir, o desfecho já registrado e a última configuração salva.
2. O plugin aplica a configuração salva e, se for o caso, mostra o convite.
3. Cada ajuste do estudante (ligar um recurso, aceitar ou recusar o convite, restaurar o padrão...) sai **na hora**: como evento no navegador e, se configurado, como webhook assinado para o backend da plataforma.
4. A plataforma guarda a configuração mais recente. No próximo acesso, em qualquer aparelho ou navegador, ela devolve essa configuração no `setUser` e o estudante encontra tudo como deixou.

> Nesse modo a Luumu **não guarda** lista de convidados nem preferências dos estudantes. Vale sempre a última configuração aplicada pelo estudante: guarde a do evento mais recente e envie de volta no `setUser`.

2. Configuração (uma vez)

No painel da Luumu (Acessibilidade)

- **Ativação > Integração**: escolha **Plataforma externa**.
- **Ativação > Integração > Webhook** (recomendado): informe a URL https do backend da plataforma e clique em **Testar webhook** (deve responder 2xx).
- **Ativação > Verificação de identidade**: copie o **Segredo do projeto** e guarde no backend da plataforma (variável de ambiente). Ele confere a assinatura do webhook e, se a verificação for exigida, gera o `user_hash`.
- **Convite > Mostrar o convite**: ligado. Mensagem, ilustração e botões do convite continuam sendo configurados no painel.
- Clique em **Publicar**.

Na página da plataforma

O SDK da Luumu precisa estar instalado (o mesmo script de sempre, antes do </head>):

```
<script src="https://luumu.com.br/sdk.js" data-luumu="SUA_CHAVE_PK" defer></script>
```

Como o script usa defer, espere o SDK antes de chamá-lo:

```
function comLuumu(fn) {
  if (window.Luumu) fn(window.Luumu);
  else window.addEventListener("luumu:ready", () => fn(window.Luumu), { once: true });
}
```

3. Entrada: o que a plataforma envia ao SDK

Uma chamada por estudante logado, com todos os campos:

```
comLuumu((Luumu) => {
  Luumu.accessibility.setUser({
    id: "123456",                    // ID estável do estudante
    email: "ana@escola.sp.gov.br",   // opcional
    user_hash: "9f2c...",            // só com "Exigir verificação"
    invite: true,                    // mostrar o convite
    profiles: ["leitura", "foco"],   // perfis do convite
    status: "pending",               // desfecho já registrado
    // última configuração salva (seção 4)
    prefs: { textSize: 2, legibleFont: true, spacing: true, profiles: ["leitura"] },
  });
});
```

Campo	Tipo	Obrigatório	O que é
`id`	texto ou número	sim*	Identificador estável do estudante. É ele que aparece em user.id nos eventos. *id ou email.
`email`	texto	sim*	E-mail do estudante. *id ou email.
`user_hash`	texto (hex)	se exigido	HMAC-SHA256 (segredo do projeto, id em minúsculas e sem espaços), em hexadecimal. Gerado no backend, nunca na página.
`invite`	booleano	sim	true = este estudante recebe o convite. false = não recebe (o botão de acessibilidade continua disponível).
`profiles`	lista de ids	não	Perfis sugeridos no convite, na ordem (ver seção 5). Até 3 aparecem. Vazio = os perfis configurados no painel.
`status`	texto	não	"pending" (não viu) | "shown" (viu e não respondeu) | "accepted" | "dismissed". Com "accepted" ou "dismissed" o convite não aparece de novo. Padrão: "pending".
`prefs`	objeto | null	não	Última configuração salva (formato da seção 4). Omitido ou null = estudante sem nada salvo ainda. Campos ausentes = padrão (desligado).

Quando chamar (gatilhos)

- **A cada carregamento de página com o estudante logado**, o quanto antes. Pode ser antes de o plugin terminar de carregar: o SDK guarda e entrega. O plugin espera até 10 segundos pelo **`setUser`** antes de desistir do convite naquela página.
- **Logo após o login sem recarregar a página** (aplicações de página única).
- **Quando algo mudar na plataforma** para esse estudante (por exemplo, passou a ser convidado). Pode chamar de novo a qualquer momento; o último valor vale.
- **No logout**: chame **`Luumu.reset()`**. As escolhas e o convite do estudante anterior saem da tela (importante em computadores compartilhados).

Regras do convite

O convite aparece somente quando todas as condições abaixo são verdadeiras:

- `invite: true` e `status` igual a `pending` ou `shown`;
- o convite está ligado no painel da Luumu e o plugin está ativo para essa plataforma (domínio);
- o estudante ainda não tem nenhum recurso ligado;
- ainda não apareceu nesta sessão do navegador (no máximo uma vez por sessão).

Os perfis do convite já aparecem marcados. O estudante só aceita (liga todos) ou recusa. Os dois desfechos voltam para a plataforma na hora (seção 6).

Como o prefs recebido é aplicado

- O plugin aplica o `prefs` recebido imediatamente e a página já abre com a configuração do estudante.
- Proteção contra cópia antiga: se o estudante mudou algo nesta página depois do último `setUser`, o próximo `setUser` não sobrescreve as escolhas (evita desfazer a mudança com um valor que ainda não incluía o ajuste). Os outros campos valem normalmente.

4. Formato da configuração (prefs)

O mesmo formato vale para a entrada (setUser) e para a saída (eventos). Guarde o objeto inteiro como veio no evento mais recente.

Campo	Tipo	Valores	Recurso
`readingMask`	booleano	true | false	Máscara de leitura
`readingGuide`	booleano	true | false	Guia de leitura
`textReader`	booleano	true | false	Leitor de texto
`legibleFont`	booleano	true | false	Fonte legível
`spacing`	booleano	true | false	Espaçamento
`textSize`	número	0 a 4 (100%, 115%, 130%, 150%, 175%)	Tamanho do texto
`contrast`	texto	"off" | "high" | "dark"	Contraste
`saturation`	texto	"off" | "low" | "high" | "mono"	Saturação
`colorBlind`	texto	"off" | "protan" | "deutan" | "tritan"	Modo daltonismo
`highlightLinks`	booleano	true | false	Destacar links
`highlightHeadings`	booleano	true | false	Destacar títulos
`bigCursor`	booleano	true | false	Cursor grande
`stopAnimations`	booleano	true | false	Pausar animações
`focusRing`	booleano	true | false	Navegação por teclado
`libras`	booleano	true | false	Libras
`profiles`	lista	ids de perfil (ver tabela de perfis)	Perfis ligados
`launcherPos`	objeto | null	{ x: 0–1, y: 0–1 }	Onde o estudante deixou o botão (fração da tela)

5. Perfis (ids)

Use os ids em `profiles` (setUser) e leia-os em `activeProfiles` e `prefs.profiles` (eventos). O título é o que o estudante vê no convite; o nome é o que aparece no painel lateral.

id	Título no convite	Nome no painel	O que liga
`ouvir`	Ouvir o conteúdo	Usuário cego (leitor de tela)	Ativa o leitor de texto e a navegação por teclado.
`visao`	Enxergar melhor	Baixa visão	Amplia o texto, sublinha links e títulos, aumenta o contraste e o cursor.
`ritmo`	Uma navegação no seu ritmo	Habilidades motoras	Ativa a navegação por teclado, o cursor grande e destaca os links.
`cores`	Cores fáceis de distinguir	Daltonismo	Ativa o modo daltonismo e destaca os links.
`calma`	Tela mais calma	Epilepsia	Diminui a saturação e bloqueia as animações.
`foco`	Mais foco	TDAH	Bloqueia as animações e ativa a máscara e o guia de leitura.
`leitura`	Mais conforto na leitura	Dislexia	Fonte legível, texto maior e mais espaço entre letras e linhas.
`libras`	Conteúdo em Libras	Deficiência auditiva	Ativa a tradução dos textos em Libras (VLibras).

6. Saída: mudanças em tempo real

Toda mudança feita no plugin gera um evento, no mesmo instante, por dois canais com o mesmo conteúdo e o mesmo id:

6.1 No navegador (front-end da plataforma)

```
window.addEventListener("luumu:accessibility", (e) => {
  const evento = e.detail;              // formato da tabela 6.4
  // ex.: enviar ao backend da plataforma
});

// ou, com o SDK carregado:
const parar = Luumu.accessibility.onChange((evento) => { /* ... */ });
// parar();                             // deixa de ouvir

// estado atual: { user, prefs, activeProfiles } (null antes de carregar)
Luumu.accessibility.getState();
```

6.2 Webhook (backend da plataforma)

Com a URL configurada no painel, a Luumu repassa cada evento para ela. Recomendado como fonte principal: chega mesmo se a página da plataforma não tratar o evento do navegador.

Item	Valor
`Método / corpo`	POST, JSON (application/json), formato da tabela 6.4
`X-Luumu-Event`	tipo do evento (ex.: prefs.updated)
`X-Luumu-Delivery`	id do evento (o mesmo do campo id do corpo)

Item	Valor
`X-Luumu-Timestamp`	segundos desde 1970 (UTC) do envio
`X-Luumu-Signature`	sha256=<hex de HMAC-SHA256(segredo do projeto, "<timestamp>.<corpo bruto>")>
`User-Agent`	Luumu-Webhooks/1.0
`Resposta esperada`	2xx em até 5 segundos. Responda primeiro e processe depois.
`Novas tentativas`	até 3 envios (na hora, +1 s, +4 s) em caso de falha de rede, timeout, 408, 429 ou 5xx. Outros 4xx não são repetidos.
`Quem gera eventos`	só estudantes identificados (id ou email). Com "Exigir verificação", só com user_hash válido.

6.3 Tipos de evento (gatilhos)

event	Quando acontece
`prefs.updated`	Qualquer ajuste: ligar ou desligar um recurso, mudar o tamanho do texto, contraste, saturação ou daltonismo, ligar ou desligar um perfil, Restaurar padrão (tudo desligado), mover o botão de lugar e aceitar o convite (que liga os perfis). O campo changes diz o que mudou.
`invite.shown`	O convite apareceu para o estudante.
`invite.accepted`	O estudante aceitou o convite. Em seguida chega um prefs.updated com os perfis ligados.
`invite.dismissed`	O estudante recusou o convite.
`test`	Enviado pelo botão Testar webhook do painel. Pode ser ignorado.

6.4 Campos do evento

Campo	Tipo	O que é
`event`	texto	Tipo do evento (tabela 6.3).
`id`	texto	Id único do evento. Use para descartar repetidos (novas tentativas mandam o mesmo id).
`occurredAt`	texto ISO 8601	Quando aconteceu no aparelho do estudante. Use para ordenar: vale o mais recente.
`user.id`	texto | null	ID do estudante (como veio no setUser / identify).
`user.email`	texto | null	E-mail do estudante, se informado.
`host`	texto	Domínio da plataforma onde aconteceu (ex.: app.escola.com.br).
`prefs`	objeto	Configuração completa do estudante já com a mudança (seção 4). É o que deve ser salvo.
`activeProfiles`	lista de ids	Perfis ligados no momento (igual a prefs.profiles).
`changes`	lista	O que mudou neste evento: [{ field, from, to }]. Vazio nos eventos de convite.
`invite`	objeto | null	Nos eventos invite.*: { status, profiles } com o desfecho e os perfis mostrados. null nos demais.

Exemplo (corpo do webhook):

```
{
  "event": "prefs.updated",
  "id": "5b0e8c2a-3f7d-4d55-9a1e-2c41f0b6a9d3",
  "occurredAt": "2026-11-12T14:03:27.512Z",
  "user": { "id": "123456", "email": "ana@escola.sp.gov.br" },
  "host": "app.escola.com.br",
  "prefs": { "readingMask": false, "readingGuide": false, "textReader": false,
             "legibleFont": true, "spacing": true, "textSize": 2, "contrast": "off",
             "saturation": "off", "colorBlind": "off", "highlightLinks": true,
             "highlightHeadings": false, "bigCursor": false, "stopAnimations": false,
             "focusRing": false, "libras": false, "profiles": ["leitura"],
             "launcherPos": null },
  "activeProfiles": ["leitura"],
  "changes": [{ "field": "highlightLinks", "from": false, "to": true }],
  "invite": null
}
```

7. Conferir a assinatura do webhook

1. Leia o corpo **bruto** (antes de converter de JSON).
2. Calcule HMAC-SHA256 com o segredo do projeto sobre **`<X-Luumu-Timestamp>.<corpo bruto>`** e compare com **`X-Luumu-Signature`** (sem o prefixo sha256=), em tempo constante.
3. Recuse se o timestamp tiver mais de 5 minutos de diferença do relógio do servidor.
4. Descarte eventos com **`id`** já processado.

```
// Node.js (Express)
import crypto from "node:crypto";
const bruto = express.raw({ type: "application/json" });
app.post("/webhooks/luumu-acessibilidade", bruto, (req, res) => {
  const ts = req.get("X-Luumu-Timestamp");
  const sig = (req.get("X-Luumu-Signature") || "").replace(/^sha256=/, "");
  const esperado = crypto.createHmac("sha256", process.env.LUUMU_A11Y_SECRET)
    .update(`${ts}.${req.body}`).digest("hex");
  const ok = sig.length === esperado.length &&
    crypto.timingSafeEqual(Buffer.from(sig), Buffer.from(esperado)) &&
    Math.abs(Date.now() / 1000 - Number(ts)) < 300;
  if (!ok) return res.sendStatus(401);
  res.sendStatus(204);                        // responda logo
  const evento = JSON.parse(req.body);
  salvarSeMaisRecente(evento);                // ver seção 8
});
```

```
# Python
import hmac, hashlib, time
def assinatura_valida(corpo: bytes, ts: str, assinatura: str, segredo: str) -> bool:
    msg = f"{ts}.".encode() + corpo
    esperado = hmac.new(segredo.encode(), msg, hashlib.sha256).hexdigest()
    return hmac.compare_digest(assinatura.removeprefix("sha256="), esperado) \
        and abs(time.time() - int(ts)) < 300
```

8. O que a plataforma deve guardar

Ao receber	Faça
`prefs.updated`	Se occurredAt for mais recente que o salvo para esse estudante, substitua a configuração salva por prefs (objeto inteiro). Ela volta no próximo setUser como prefs.
`invite.shown`	Registre status "shown" (se ainda for "pending").
`invite.accepted`	Registre status "accepted". Não volta atrás.
`invite.dismissed`	Registre status "dismissed" (se não for "accepted").

Um estudante pode usar vários aparelhos ao mesmo tempo: comparar occurredAt garante que vale a última necessidade aplicada por ele.

9. Checklist

- SDK instalado com a chave do projeto em todas as páginas logadas.
- Painel: Integração = Plataforma externa, convite ligado, webhook testado (2xx) e alterações publicadas.
- `setUser` chamado a cada página com id, invite, profiles, status e prefs salvos.
- `Luumu.reset()` no logout.
- Webhook confere assinatura e janela de 5 minutos, responde 2xx em até 5 s e descarta ids repetidos.
- Configuração salva só quando occurredAt for mais recente; status do convite atualizado nos eventos invite.*.
- Teste ponta a ponta: estudante convidado vê o convite, aceita; em outro navegador ou aparelho, a configuração aparece aplicada e o convite não volta.